Proyectos de Programación 26/27
Práctica de CMake – 26.10.2026

En esta práctica vas a entender **qué pasa de verdad cuando "compilas"** un programa en C++ y por qué en los proyectos reales nadie escribe a mano los comandos de compilación, sino que usa **CMake**. Al terminar, sabrás compilar y ejecutar un programa de varios ficheros, leer los dos tipos de error más habituales (el del **compilador** y el del **enlazador**) y montar un proyecto con la estructura profesional `include/` + `src/` + `CMakeLists.txt`: exactamente la que necesita vuestro proyecto.

No tienes que traer nada: todo el código del que partimos está en un repositorio público de GitHub que vas a descargar en el paso 1.2. Sigue la guía **en orden**, sin saltarte pasos: cada uno usa lo que has hecho en el anterior.

Cómo leer esta guía

- Los recuadros negros son la **terminal**. Lo que tienes que escribir está en `amarillo`; lo que va delante (`PS C:\Users\alumno>` o `alumno@MacBook … %`) es el *prompt*: lo pone la terminal, **no lo escribas**. Lo que aparece en gris debajo es lo que responde el ordenador.
- Cuando Windows y Mac son distintos verás dos recuadros, uno al lado del otro: **usa solo el de tu sistema**.
- En tu ordenador, en lugar de `alumno` aparecerá tu nombre de usuario. Es normal.
- Después de escribir cada comando, pulsa Enter.
- Los recuadros verdes dicen lo que deberías ver; los rojos, qué hacer si ves otra cosa.

0. La terminal sin miedo (15 min)

La terminal es una ventana donde, en lugar de hacer clic, **escribes órdenes**. Siempre está "situada" en una carpeta de tu ordenador, igual que el Explorador de archivos o el Finder: todo lo que hagas (compilar, crear carpetas…) ocurre **en esa carpeta**. El 90 % de los problemas de esta práctica se resuelven sabiendo en qué carpeta estás.

0.1 Abrir una terminal

- **Windows:** pulsa la tecla Windows, escribe **PowerShell** y pulsa Enter sobre "Windows PowerShell".
- **Mac:** pulsa ⌘ Cmd + Espacio, escribe **Terminal** y pulsa Enter.
- **Dentro de VS Code** (lo usaremos a partir del paso 1.3): menú **Terminal → Nueva terminal**. Se abre abajo, y es exactamente igual que las anteriores.

0.2 ¿Dónde estoy? → pwd

Escribe `pwd` (*print working directory*, "muestra la carpeta actual"):

Windows · PowerShell

```
PS C:\Users\alumno> pwd

Path
----
C:\Users\alumno
```

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % pwd
/Users/alumno
```


Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

> **¿Qué ha pasado?**
> Al abrir la terminal siempre empiezas en tu **carpeta personal**. En Windows el prompt ya te dice la carpeta (`PS C:\Users\alumno>`). En Mac el prompt solo muestra el **último nombre** de la ruta, y la carpeta personal se abrevia con `~` . **Si alguna vez dudas de dónde estás, escribe `pwd` .**

0.3 ¿Qué hay aquí? → ls

Windows · PowerShell

```
PS C:\Users\alumno> ls
    Directorio: C:\Users\alumno

Mode        LastWriteTime     Length Name
----        -------------     ------ ----
d-r---      01/09/2026 10:12         Desktop
d-r---      01/09/2026 10:12         Documents
d-r---      14/10/2026 18:40         Downloads
```

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % ls
Applications  Desktop  Documents  Downloads  Movies  Music  Pictures
```

Verás las carpetas de tu usuario (en Windows pueden aparecer en inglés: *Desktop* = Escritorio, *Documents* = Documentos…).

0.4 Crear una carpeta para la asignatura → mkdir

Vamos a crear una carpeta llamada `PP` dentro de tu carpeta personal. Ahí guardarás todo lo de la asignatura.

Windows · PowerShell

```
PS C:\Users\alumno> mkdir PP

    Directorio: C:\Users\alumno

Mode        LastWriteTime     Length Name
----        -------------     ------ ----
d-----      26/10/2026 15:05         PP
```

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % mkdir PP
```

En Mac, si todo va bien, **no aparece nada**: en la terminal, "no decir nada" significa "ha ido bien".

> **✘ Si en vez de eso ves…**
> `mkdir : Ya existe un elemento con el nombre especificado` (Windows) o `mkdir: PP: File exists` (Mac): la carpeta ya existía (quizá de la práctica de Git). No pasa nada: sigue al paso siguiente.

0.5 Entrar en una carpeta → cd · Volver atrás → cd ..

`cd` (*change directory*) te mueve a otra carpeta. Entra en `PP` y comprueba dónde estás:

Windows · PowerShell

```
PS C:\Users\alumno> cd PP
PS C:\Users\alumno\PP> pwd

Path
----
C:\Users\alumno\PP
```

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % cd PP
alumno@MacBook-de-alumno PP % pwd
/Users/alumno/PP
```

Fíjate en que **el prompt ha cambiado**: ahora termina en `PP`. Para volver a la carpeta de arriba se usa `cd ..` (dos puntos = "la carpeta que contiene a esta"). Pruébalo y vuelve a entrar:

Windows · PowerShell

```
PS C:\Users\alumno\PP> cd ..
PS C:\Users\alumno> cd PP
PS C:\Users\alumno\PP>
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % cd ..
alumno@MacBook-de-alumno ~ % cd PP
alumno@MacBook-de-alumno PP %
```

Truco

Autocompletar con Tab: escribe solo el principio de un nombre (por ejemplo `cd pa`) y pulsa Tab: la terminal completa el resto. Evita errores de escritura y es lo que hacen todos los profesionales.
Repetir un comando: la flecha ↑ recupera los comandos anteriores.

0.6 Resumen de la parte 0

Quiero...	Comando	Ejemplo
Saber en qué carpeta estoy	`pwd`	`pwd`
Ver qué hay en la carpeta	`ls`	`ls`
Crear una carpeta	`mkdir nombre`	`mkdir PP`
Entrar en una carpeta	`cd nombre`	`cd PP`
Subir a la carpeta de arriba	`cd ..`	`cd ..`
Ir directamente a mi carpeta personal	`cd ~`	`cd ~`

Atención

Usa nombres de carpeta **sin espacios ni tildes** (`PP`, `pp-lab2-cmake`...). Con espacios, muchos comandos fallan de formas raras. Y en Windows, **no trabajes dentro de OneDrive** (por ejemplo, en *Documentos* si está sincronizado): usa la carpeta `C:\Users\tu-usuario\PP` que acabas de crear.

Pirineos, 55 - 28040 Madrid - Tel. 91 444 51 61 www.cunef.edu

1. Preparar el equipo y descargar el código (10 min)

1.1 Comprobar que tienes las herramientas

Las instalaste en la Práctica 0. Comprueba que siguen ahí (da igual en qué carpeta estés). Cada comando debe responder con un número de versión, **no con un error**:

Windows · PowerShell

```
PS C:\Users\alumno\PP> g++ --version
g++.exe (Rev2, Built by MSYS2 project) 14.2.0
PS C:\Users\alumno\PP> cmake --version
cmake version 3.31.0
PS C:\Users\alumno\PP> git --version
git version 2.51.0.windows.2
PS C:\Users\alumno\PP> ninja --version
1.12.1
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % g++ --version
Apple clang version 17.0.0 (clang-1700.0.13.3)
alumno@MacBook-de-alumno PP % cmake --version
cmake version 4.1.2
alumno@MacBook-de-alumno PP % git --version
git version 2.39.5 (Apple Git-154)
```

Los números exactos pueden ser distintos: lo importante es que aparezca una versión. (En Mac, `g++` es en realidad el compilador *clang* de Apple; funciona igual. **En Mac no hace falta `ninja`.**)

> **Atención**
>
> **¿Te falta alguna herramienta?** Al final de esta guía tienes el **Anexo · Instalar las herramientas**, con un repaso de la Práctica 0 para Windows y para Mac (desde dónde se descarga cada cosa y qué comandos escribir). Instalar lleva unos 15-20 minutos y necesita internet: mientras tanto, trabaja con el ordenador de tu pareja.

> **✘ Si en vez de eso ves…**
>
> **Windows, `ninja` no se reconoce** (`ninja : El término 'ninja' no se reconoce como nombre de un cmdlet…`): Ninja es el programa que CMake usará en Windows para compilar. Instálalo así:
>
> 1. Abre la terminal **MSYS2 MSYS** (tecla Windows → escribe MSYS2 → "MSYS2 MSYS"), la misma de la Práctica 0.
> 2. Escribe `pacman -S --needed mingw-w64-x86_64-ninja` y confirma con `Y`.
> 3. Cierra **todas** las ventanas de PowerShell, abre una nueva y repite `ninja --version`.
>
> **Windows, falla `g++` o `cmake`** ("no se reconoce como nombre de un cmdlet"):
>
> - Si **no llegaste a instalar** MSYS2, g++ o CMake: **Anexo, Windows, pasos W1-W3**.
> - Si los instalaste, casi siempre falta añadir `C:\msys64\mingw64\bin` al PATH: **Anexo, paso W3**. Después cierra y abre la terminal.
>
> **Windows, falla `git`**: **Anexo, paso W4**.
>
> **Mac, `zsh: command not found: cmake`**: o no instalaste CMake, o instalaste la aplicación pero te faltó el paso *How to Install For Command Line Use*. Las dos cosas están en el **Anexo, paso M2**.
>
> **Mac, `xcrun: error: invalid active developer path`**, o una ventana que te pide instalar las "herramientas de desarrollo": escribe `xcode-select --install` y acepta (**Anexo, paso M1**).

1.2 Descargar (clonar) el repositorio de la práctica

Asegúrate de estar en tu carpeta `PP` (mira el prompt o haz `pwd`). Después descarga el repositorio con `git clone`:

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/jcor-cunpir/pp-lab2-cmake.git
Cloning into 'pp-lab2-cmake'...
remote: Enumerating objects: …, done.
Receiving objects: 100% (…), done.
PS C:\Users\alumno\PP> cd pp-lab2-cmake
PS C:\Users\alumno\PP\pp-lab2-cmake> ls

    Directorio: C:\Users\alumno\PP\pp-lab2-cmake

Mode        LastWriteTime     Length Name
----        -------------     ------ ----
d-----      26/10/2026 15:10         paso-01-un-fichero
d-----      26/10/2026 15:10         paso-02-varios-ficheros
d-----      26/10/2026 15:10         paso-03-primer-cmake
d-----      26/10/2026 15:10         paso-04-estructura
d-----      26/10/2026 15:10         paso-05-ejercicio
d-----      26/10/2026 15:10         paso-05-solucion
-a----      26/10/2026 15:10  373771 Practica CMake - 2026.10.26.pdf
-a----      26/10/2026 15:10    1738 README.md
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/jcor-cunpir/pp-lab2-cmake.git
Cloning into 'pp-lab2-cmake'...
remote: Enumerating objects: …, done.
Receiving objects: 100% (…), done.
alumno@MacBook-de-alumno PP % cd pp-lab2-cmake
alumno@MacBook-de-alumno pp-lab2-cmake % ls
Practica CMake - 2026.10.26.pdf
README.md
paso-01-un-fichero
paso-02-varios-ficheros
paso-03-primer-cmake
paso-04-estructura
paso-05-ejercicio
paso-05-solucion
```

Cada carpeta `paso-…` es un paso de esta guía. **Si te atascas en un paso, puedes pasar al siguiente**: cada carpeta empieza ya resuelta respecto a la anterior.

✘ Si en vez de eso ves…

`fatal: destination path 'pp-lab2-cmake' already exists`: ya lo habías descargado. Entra en la carpeta con `cd pp-lab2-cmake` y sigue.

`git` no funciona: descarga el código desde el navegador en **https://github.com/jcor-cunpir/pp-lab2-cmake** → botón verde **Code** → **Download ZIP**, descomprímelo dentro de tu carpeta `PP` y renombra la carpeta a `pp-lab2-cmake`. Después, en casa, instala Git (**Anexo, paso W4 o M1**): lo necesitarás para la plantilla del proyecto.

1.3 Abrir el proyecto en VS Code

Desde la carpeta `pp-lab2-cmake`, escribe `code .` (el punto significa “esta carpeta”):

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\pp-lab2-cmake> code .
```

1. Se abre VS Code con la carpeta del proyecto. Si pregunta "¿Confía en los autores de los archivos de esta carpeta?", pulsa **Sí, confío en los autores**.
2. A la izquierda (el **Explorador**) verás las carpetas `paso-01…` a `paso-05…`. Haz clic en una para ver sus ficheros; doble clic en un fichero para abrirlo.
3. Abre la terminal integrada: menú **Terminal → Nueva terminal**. Escribe `pwd`: debe terminar en `pp-lab2-cmake`. **A partir de aquí, usa esta terminal.**

✘ Si en vez de eso ves…

Mac, `zsh: command not found: code`: abre VS Code a mano, pulsa ⌘ Cmd + ⇧ Shift + P, escribe **shell command** y elige **Shell Command: Install 'code' command in PATH**. Cierra la terminal, abre una nueva y repite `code .`

Windows, `code` no se reconoce, o si lo anterior no funciona: abre VS Code a mano, menú **Archivo → Abrir carpeta…** y elige la carpeta `PP/pp-lab2-cmake`. Después abre la terminal con **Terminal → Nueva terminal**.

Paso 1 · Compilar un programa de un solo fichero (10 min)

Entra en la carpeta del paso 1 y mira qué contiene:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake> cd paso-01-un-fichero
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-01-un-fichero> ls
...
-a----    26/10/2026 15:10    271 main.cpp
```

Mac · Terminal

```
alumno@MacBook-de-alumno pp-lab2-cmake % cd paso-01-un-fichero
alumno@MacBook-de-alumno paso-01-un-fichero % ls
main.cpp
```

Abre `main.cpp` en VS Code (Explorador → `paso-01-un-fichero` → `main.cpp`). Es un programa normal:

paso-01-un-fichero/main.cpp

```
// PASO 1 — Un programa de un solo fichero.
// Compilar:  g++ main.cpp -o hotel
// Ejecutar:  ./hotel        (Mac)
//            .\hotel.exe    (Windows)
#include <iostream>

int main() {
    std::cout << "Bienvenido al Hotel CUNEF\n";
    std::cout << "Habitaciones libres: 3\n";
    return 0;
}
```

El ordenador no entiende C++: solo entiende código máquina. El **compilador** (`g++`) traduce tu `main.cpp` a un **ejecutable**. Escribe:

Windows y Mac (igual en los dos)

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-01-un-fichero> g++ main.cpp -o hotel
```

Trozo	Significa
`g++`	el programa compilador
`main.cpp`	el fichero que quieres compilar
`-o hotel`	"**o**utput": llama `hotel` al ejecutable que generes

Si no aparece nada, ha ido bien. Comprueba con `ls` que hay un fichero nuevo y **ejecútalo**:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-01-un-fichero> ls
...
-a----    26/10/2026 15:21  152311 hotel.exe
-a----    26/10/2026 15:10     271 main.cpp
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-01-un-fichero> .\hotel.exe
Bienvenido al Hotel CUNEF
Habitaciones libres: 3
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-01-un-fichero % ls
hotel  main.cpp
alumno@MacBook-de-alumno paso-01-un-fichero % ./hotel
Bienvenido al Hotel CUNEF
Habitaciones libres: 3
```

¿Qué ha pasado?

Para ejecutar un programa que está en la carpeta actual hay que escribir `.\` (Windows) o `./` (Mac) delante: significa "el programa que está **aquí**". En Windows el ejecutable termina en `.exe` ; en Mac no lleva extensión.

✘ Si en vez de eso ves...

`cc1plus: fatal error: main.cpp: No such file or directory`: no estás en la carpeta del paso 1. Haz `pwd` y `ls` : tienes que ver `main.cpp` en la lista.

`.\hotel.exe : El término … no se reconoce` / `zsh: command not found: hotel` : te has olvidado del `.\` o `./` del principio, o el comando `g++` anterior dio un error.

Paso 2 · Varios ficheros: compilar y enlazar (25 min)

Los programas de verdad tienen muchas clases, y cada clase va en **dos ficheros**. Vuelve a la carpeta de arriba y entra en la del paso 2:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-01-un-fichero> cd ..
PS C:\Users\alumno\PP\pp-lab2-cmake> cd paso-02-varios-ficheros
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> ls
...
habitacion.cpp
habitacion.hpp
main.cpp
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-01-un-fichero % cd ..
alumno@MacBook-de-alumno pp-lab2-cmake % cd paso-02-varios-ficheros
alumno@MacBook-de-alumno paso-02-varios-ficheros % ls
habitacion.cpp  habitacion.hpp  main.cpp
```

Ábrelos los tres en VS Code y compáralos:

Fichero	Qué contiene	Analogía
`habitacion.hpp`	La **declaración** de la clase: qué métodos tiene (sin su código)	El manual de instrucciones del microondas
`habitacion.cpp`	La **implementación**: el código de cada método (`Habitacion::getNumero()` …)	El circuito interno
`main.cpp`	El programa que **usa** la clase. Solo hace `#include "habitacion.hpp"`	Quien usa el microondas

2.1 Primer intento: compilar solo main.cpp

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ main.cpp -o hotel
…/ld.exe: …main.cpp:(.text+0x34): undefined reference to `Habitacion::Habitacion(int, double)'
…/ld.exe: …main.cpp:(.text+0x6c): undefined reference to `Habitacion::getNumero() const'
…/ld.exe: …main.cpp:(.text+0x98): undefined reference to `Habitacion::getPrecio() const'
…/ld.exe: …main.cpp:(.text+0x138): undefined reference to `Habitacion::getTotal()'
collect2.exe: error: ld returned 1 exit status
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ main.cpp -o hotel
Undefined symbols for architecture arm64:
  "Habitacion::Habitacion(int, double)", referenced from:
      _main in main-3f1a2b.o
  "Habitacion::getNumero() const", referenced from:
      _main in main-3f1a2b.o
  …
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1
```

(El texto exacto puede variar un poco según la versión del compilador; en un Mac con procesador Intel pone `x86_64` en vez de `arm64`.)

> **¿Qué ha pasado?**
>
> **Es un error del ENLAZADOR, no del compilador.** Léelo así:
>
> - `ld` es el **enlazador** (en inglés, *linker*). `ld returned 1 exit status` (Windows) o `linker command failed` (Mac) = "el enlazador ha fallado".
> - `undefined reference to `Habitacion::getNumero() const'` (Windows) o `Undefined symbols … "Habitacion::getNumero() const"` (Mac) = "`main` usa `getNumero()`, pero **no encuentro su código** en ningún sitio".
> - El compilador sí estaba contento: `main.cpp` incluye `habitacion.hpp`, y ahí **se promete** que esos métodos existen. Pero su código está en `habitacion.cpp`, y ese fichero **no se lo hemos dado** a `g++`.
>
> En Windows, delante de cada línea aparece una ruta muy larga (`C:/msys64/mingw64/…/ld.exe: …`). Ignórala: lo importante es lo que va después de `undefined reference to`.

2.2 Segundo intento: darle los dos .cpp

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ main.cpp habitacion.cpp -o ho
tel
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> .\hotel.exe
Habitacion 101: 80 EUR
Habitacion 102: 95.5 EUR
Total de habitaciones: 2
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ main.cpp habitacion.cpp -o hotel
alumno@MacBook-de-alumno paso-02-varios-ficheros % ./hotel
Habitacion 101: 80 EUR
Habitacion 102: 95.5 EUR
Total de habitaciones: 2
```

Los `.hpp` **no se ponen** en el comando: ya los meten los `#include` de los `.cpp`.

2.3 Lo que g++ hace por dentro: compilar y enlazar por separado

El comando anterior hace **dos cosas** seguidas. Vamos a hacerlas a mano, una a una. Primero **compilar** cada `.cpp` por separado con la opción `-c` ("solo compilar, no enlazar"):

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ -c habitacion.cpp
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ -c main.cpp
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> ls
...
habitacion.cpp
habitacion.hpp
habitacion.o
hotel.exe
main.cpp
main.o
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ -c habitacion.cpp
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ -c main.cpp
alumno@MacBook-de-alumno paso-02-varios-ficheros % ls
habitacion.cpp  habitacion.o  main.cpp
habitacion.hpp  hotel         main.o
```

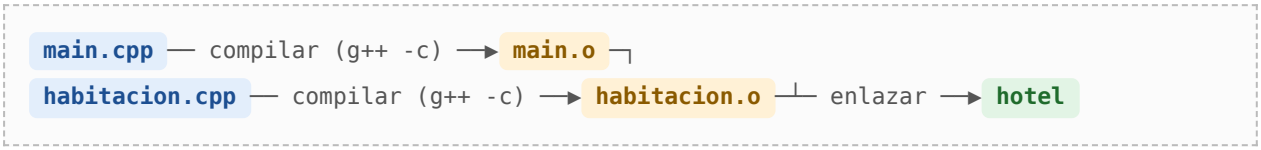
Han aparecido `habitacion.o` y `main.o`: son **ficheros objeto**, código máquina de cada `.cpp` por separado, todavía "con huecos" (`main.o` llama a `getNumero()` pero no sabe dónde está). Ahora, **enlazar**: unir los `.o` y rellenar esos huecos:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ main.o habitacion.o -o hotel
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> .\hotel.exe
Habitacion 101: 80 EUR
Habitacion 102: 95.5 EUR
Total de habitaciones: 2
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ main.o habitacion.o -o hotel
alumno@MacBook-de-alumno paso-02-varios-ficheros % ./hotel
Habitacion 101: 80 EUR
Habitacion 102: 95.5 EUR
Total de habitaciones: 2
```



2.4 Un error del COMPILADOR (para comparar)

1. En VS Code, abre `paso-02-varios-ficheros/main.cpp`.
2. En la línea 8 cambia `h1.getNumero()` por `h1.getNumro()` (quita la segunda **e**).
3. **Guarda el fichero** con Ctrl+S (Mac: ⌘ Cmd+S). Si en la pestaña del fichero ves un punto blanco ●, es que **no** está guardado.
4. Compila solo `main.cpp`:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ -c main.cpp
main.cpp: In function 'int main()':
main.cpp:8:38: error: 'class Habitacion' has no member named 'getNumro'; did you mean 'getNume
ro'?
    8 |     std::cout << "Habitacion " << h1.getNumro() << …
      |                                    ^~~~~~~
      |                                    getNumero
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ -c main.cpp
main.cpp:8:38: error: no member named 'getNumro' in 'Habitacion'; did you mean 'getNumero'?
    8 |     std::cout << "Habitacion " << h1.getNumro() << …
      |                                    ^~~~~~~~
      |                                    getNumero
1 error generated.
```

> **¿Qué ha pasado?**
> Ahora el error es del **compilador**: dice el **fichero, la línea y la columna** (`main.cpp:8:38`) e incluso sugiere la solución. Los errores del enlazador (2.1) **no** tienen número de línea, porque el problema no está en una línea concreta, sino en que falta un trozo del programa.

Deshaz el cambio (vuelve a escribir `getNumero`) y **guarda**.

2.5 Un static declarado pero no definido

1. Abre `habitacion.cpp`. En la línea 4 está la definición del atributo `static`: `int Habitacion::total_ = 0;`
2. Coméntala poniendo `//` delante: `// int Habitacion::total_ = 0;` y **guarda**.
3. Compila los dos ficheros:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> g++ main.cpp habitacion.cpp -o ho
tel
…/ld.exe: …habitacion.cpp:(.text+0x28): undefined reference to `Habitacion::total_'
…/ld.exe: …habitacion.cpp:(.text+0x80): undefined reference to `Habitacion::total_'
…more undefined references to `Habitacion::total_' follow
collect2.exe: error: ld returned 1 exit status
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % g++ main.cpp habitacion.cpp -o hotel
Undefined symbols for architecture arm64:
  "Habitacion::total_", referenced from:
      Habitacion::Habitacion(int, double) in habitacion-8c2d1e.o
      Habitacion::getTotal() in habitacion-8c2d1e.o
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1
```

Otra vez el **enlazador**: el `.hpp` **declara** que `total_` existe, pero nadie lo **define**. Quita el `//`, guarda y vuelve a compilar para comprobar que todo funciona.

	Error del COMPILADOR	Error del ENLAZADOR
Cuándo	Al traducir **un** `.cpp`	Al **unir** todos los `.o`
Cómo se reconoce	`fichero.cpp:línea:columna: error: …`	`undefined reference to …` (Windows) / `Undefined symbols …` (Mac), y `ld` / *linker* al final
Causa típica	Errata, falta un `;` , falta un `#include`	Falta un `.cpp` en el comando, o un método/ `static` declarado pero no definido

Paso 3 · Tu primer CMakeLists.txt (20 min)

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-02-varios-ficheros> cd ..
PS C:\Users\alumno\PP\pp-lab2-cmake> cd paso-03-primer-cmake
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> ls
...
CMakeLists.txt
habitacion.cpp
habitacion.hpp
main.cpp
reserva.cpp
reserva.hpp
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-02-varios-ficheros % cd ..
alumno@MacBook-de-alumno pp-lab2-cmake % cd paso-03-primer-cmake
alumno@MacBook-de-alumno paso-03-primer-cmake % ls
CMakeLists.txt  habitacion.hpp  reserva.cpp
habitacion.cpp  main.cpp        reserva.hpp
```

Ahora hay dos clases. A mano sería `g++ main.cpp habitacion.cpp reserva.cpp -o hotel`. Imagina vuestro proyecto con 15 clases: el comando sería larguísimo, cada compañero lo escribiría distinto, y **cada vez que cambias un fichero se recompila todo**. **CMake** resuelve eso: escribes **una vez** qué ficheros forman el programa, en un fichero llamado `CMakeLists.txt`, y CMake se encarga del resto (en Windows, en Mac y en Linux).

paso-03-primer-cmake/CMakeLists.txt

```
# CMakeLists.txt — la "receta" que CMake lee para saber cómo construir el programa.

# 1. Versión mínima de CMake que necesita este proyecto.
cmake_minimum_required(VERSION 3.15)

# 2. Nombre del proyecto y lenguaje (CXX = C++).
project(Hotel LANGUAGES CXX)

# 3. Usar C++17.
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 4. El ejecutable se llamará "hotel" y se construye con ESTOS .cpp
#    (los .hpp no se ponen: los incluyen los .cpp).
add_executable(hotel
    main.cpp
    habitacion.cpp
    reserva.cpp
)
```

3.1 Configurar (solo la primera vez)

Este comando lee el `CMakeLists.txt` y prepara una carpeta `build` donde se construirá todo. **Ojo: en Windows lleva algo más**:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> cmake -S . -B build -G Ninja
-- The CXX compiler identification is GNU 14.2.0
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: C:/msys64/mingw64/bin/c++.exe - skipped
-- Configuring done (1.9s)
-- Generating done (0.0s)
-- Build files have been written to: C:/Users/alumno/PP/pp-lab2-cmake/paso-03-primer-cmake/bui
ld
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % cmake -S . -B build
-- The CXX compiler identification is AppleClang 17.0.0
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Configuring done (0.8s)
-- Generating done (0.0s)
-- Build files have been written to: /Users/alumno/PP/pp-lab2-cmake/paso-03-primer-cmake/build
```

Trozo	Significa
`-S .`	**S**ource: el código (y el `CMakeLists.txt`) está **aquí** (el punto = carpeta actual)
`-B build`	**B**uild: deja todo lo generado en una carpeta llamada `build`
`-G Ninja` (solo Windows)	Usa **Ninja** para compilar con el g++ de MSYS2. En Mac no hace falta

✔ Deberías ver

La última línea, `-- Build files have been written to: …/build`. Si haces `ls`, ahora hay una carpeta `build`.

✘ Si en vez de eso ves…

`does not appear to contain CMakeLists.txt`: no estás en la carpeta del paso 3 (haz `pwd`).
`CMake was unable to find a build program corresponding to "Ninja"`: falta Ninja; vuelve al recuadro rojo del paso 1.1.
Windows, errores que mencionan `NMake`, `Visual Studio` o `CMAKE_CXX_COMPILER`: casi seguro que te has olvidado de `-G Ninja`. Borra la carpeta `build` (`Remove-Item -Recurse -Force build`) y repite el comando completo.
Mac, `No CMAKE_CXX_COMPILER could be found`: faltan las herramientas de Apple; escribe `xcode-select --install` (**Anexo, paso M1**).
`Does not match the generator used previously`: ya habías configurado antes de otra forma. Borra la carpeta `build` (Windows: `Remove-Item -Recurse -Force build` · Mac: `rm -rf build`) y repite el comando.

3.2 Compilar

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> cmake --build build
[1/4] Building CXX object CMakeFiles/hotel.dir/reserva.cpp.obj
[2/4] Building CXX object CMakeFiles/hotel.dir/habitacion.cpp.obj
[3/4] Building CXX object CMakeFiles/hotel.dir/main.cpp.obj
[4/4] Linking CXX executable hotel.exe
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % cmake --build build
[ 25%] Building CXX object CMakeFiles/hotel.dir/main.cpp.o
[ 50%] Building CXX object CMakeFiles/hotel.dir/habitacion.cpp.o
[ 75%] Building CXX object CMakeFiles/hotel.dir/reserva.cpp.o
[100%] Linking CXX executable hotel
[100%] Built target hotel
```

¿Reconoces los pasos? **Building … .o/.obj** = compilar cada `.cpp` (lo que hacías con `g++ -c`); **Linking** = enlazar. CMake hace lo mismo que tú en el paso 2.3, pero solo.

3.3 Ejecutar

El ejecutable está **dentro** de la carpeta `build`:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> .\build\hotel.exe
Habitaciones creadas: 2
Reserva de la 102 (3 noches): 286.5 EUR
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % ./build/hotel
Habitaciones creadas: 2
Reserva de la 102 (3 noches): 286.5 EUR
```

3.4 La gracia de CMake: solo recompila lo que cambia

Vuelve a compilar **sin cambiar nada**:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> cmake --build build
ninja: no work to do.
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % cmake --build build
[100%] Built target hotel
```

1. Ahora abre `reserva.cpp` y cambia **solo** ese fichero: por ejemplo, añade al final de la línea del `return` un comentario `// cambiado`.
2. **Guarda** (Ctrl + S) y compila otra vez:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> cmake --build build
[1/2] Building CXX object CMakeFiles/hotel.dir/reserva.cpp.obj
[2/2] Linking CXX executable hotel.exe
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % cmake --build build
[ 25%] Building CXX object CMakeFiles/hotel.dir/reserva.cpp.o
[ 50%] Linking CXX executable hotel
[100%] Built target hotel
```

¿Qué ha pasado?

Solo se ha recompilado `reserva.cpp` y se ha vuelto a enlazar. Con 3 ficheros no se nota; con un proyecto de 200 ficheros es la diferencia entre esperar 2 segundos y esperar 5 minutos. A partir de ahora, el ciclo de trabajo es siempre: **editar → guardar → `cmake --build build` → ejecutar**. El comando de configurar (3.1) solo se repite si borras `build`.

Atención

La carpeta `build` se genera sola y **nunca se sube a Git**: ya está en el fichero `.gitignore` del repositorio. Si algo se lía, se puede borrar y regenerar con los dos comandos (3.1 y 3.2).

Paso 4 · La estructura profesional: include/ y src/ (20 min)

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-03-primer-cmake> cd ..
PS C:\Users\alumno\PP\pp-lab2-cmake> cd paso-04-estructura
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-03-primer-cmake % cd ..
alumno@MacBook-de-alumno pp-lab2-cmake % cd paso-04-estructura
```

En el Explorador de VS Code despliega `paso-04-estructura`. Ahora los ficheros están ordenados como en cualquier proyecto real (y como estará el vuestro):

Estructura de carpetas

```
paso-04-estructura/
├── CMakeLists.txt
├── include/          ← las cabeceras .hpp (la interfaz)
│   ├── cliente.hpp
│   ├── habitacion.hpp
│   └── reserva.hpp
└── src/              ← las implementaciones .cpp
    ├── cliente.cpp
    ├── habitacion.cpp
    ├── main.cpp
    └── reserva.cpp
```

paso-04-estructura/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.15)
project(Hotel LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# Warnings activados: el compilador nos avisa de errores probables.
if(MSVC)
    add_compile_options(/W4)
else()
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# Los .cpp ahora viven en la carpeta src/
add_executable(hotel
    src/main.cpp
    src/habitacion.cpp
    src/reserva.cpp
)

# Los .hpp viven en la carpeta include/: hay que decírselo al compilador.
target_include_directories(hotel PRIVATE include)
```

- `-Wall -Wextra -Wpedantic`: activa los **avisos** (*warnings*) del compilador. Te avisa de cosas que compilan pero probablemente son un error.

- `src/main.cpp`…: como los `.cpp` están en `src/`, se escribe la ruta.
- `target_include_directories(hotel PRIVATE include)`: le dice al compilador "cuando veas `#include "habitacion.hpp"`, búscalo en la carpeta `include`".

4.1 Configurar y compilar… y algo falla

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> cmake -S . -B build -G Ninja
...
-- Build files have been written to: …/paso-04-estructura/build
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> cmake --build build
[1/4] Building CXX object CMakeFiles/hotel.dir/src/main.cpp.obj
…
[4/4] Linking CXX executable hotel.exe
FAILED: hotel.exe
…/ld.exe: …main.cpp:(.text+0xa4): undefined reference to `Cliente::Cliente(…)'
…/ld.exe: …main.cpp:(.text+0x104): undefined reference to `Cliente::getNombre[abi:cxx11]() con
st'
…/ld.exe: …main.cpp:(.text+0x13c): undefined reference to `Cliente::getDni[abi:cxx11]() const'
collect2.exe: error: ld returned 1 exit status
ninja: build stopped: subcommand failed.
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-04-estructura % cmake -S . -B build
...
-- Build files have been written to: …/paso-04-estructura/build
alumno@MacBook-de-alumno paso-04-estructura % cmake --build build
[ 25%] Building CXX object CMakeFiles/hotel.dir/src/main.cpp.o
…
[100%] Linking CXX executable hotel
Undefined symbols for architecture arm64:
  "Cliente::Cliente(…)", referenced from:
      _main in main.cpp.o
ld: symbol(s) not found for architecture arm64
make[2]: *** [hotel] Error 1
```

> **Piensa antes de mirar la solución**
>
> Es un error del **enlazador** (en Mac se escribe distinto: *Undefined symbols … ld: symbol(s) not found*, pero significa lo mismo). Falta el código de `Cliente`. Mira la carpeta `src/` en VS Code y compárala con la lista de `add_executable` del `CMakeLists.txt`. **¿Qué `.cpp` existe pero no está en la lista?**

4.2 Arreglarlo

1. En `CMakeLists.txt`, añade la línea `src/cliente.cpp` dentro de `add_executable(…)`, debajo de `src/reserva.cpp`.
2. **Guarda** y compila otra vez (no hace falta volver a configurar: CMake detecta que el `CMakeLists.txt` ha cambiado y se reconfigura solo).

CMakeLists.txt (arreglado)

```
add_executable(hotel
    src/main.cpp
    src/habitacion.cpp
    src/reserva.cpp
    src/cliente.cpp
)
```

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> cmake --build build
[0/2] Re-running CMake...
…
[2/3] Building CXX object CMakeFiles/hotel.dir/src/cliente.cpp.obj
[3/3] Linking CXX executable hotel.exe
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> .\build\hotel.exe
Cliente: Ana Garcia (12345678Z)
Habitacion 201, 2 noches: 240 EUR
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-04-estructura % cmake --build build
-- Configuring done
…
[ 20%] Building CXX object CMakeFiles/hotel.dir/src/cliente.cpp.o
[ 40%] Linking CXX executable hotel
[100%] Built target hotel
alumno@MacBook-de-alumno paso-04-estructura % ./build/hotel
Cliente: Ana Garcia (12345678Z)
Habitacion 201, 2 noches: 240 EUR
```

> **Atención**
>
> **Regla de oro de CMake:** cada vez que crees un `.cpp` nuevo, **añádelo al `CMakeLists.txt`**. Si se te olvida, el síntoma es siempre el mismo: `undefined reference to …` con métodos de esa clase.

4.3 Los warnings son tus amigos

1. En `src/main.cpp`, justo debajo de `Habitacion h(201, 120.0);`, añade la línea `int sinUsar = 5;` y guarda.
2. Compila:

Windows (en Mac el aviso es igual; solo cambian las líneas [25%] … del progreso)

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> cmake --build build
[1/2] Building CXX object CMakeFiles/hotel.dir/src/main.cpp.obj
…/src/main.cpp:9:9: warning: unused variable 'sinUsar' [-Wunused-variable]
    9 |     int sinUsar = 5;
      |         ^~~~~~
[2/2] Linking CXX executable hotel.exe
```

Un **warning** no impide generar el programa, pero avisa de algo sospechoso (aquí, una variable que no sirve para nada). **En vuestro proyecto, el objetivo es compilar con 0 warnings.** Borra la línea, guarda y vuelve a compilar.

Paso 5 · Ejercicio: separa una clase en .hpp y .cpp (20 min)

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-04-estructura> cd ..
PS C:\Users\alumno\PP\pp-lab2-cmake> cd paso-05-ejercicio
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-05-ejercicio> cmake -S . -B build -G Ninja
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-05-ejercicio> cmake --build build
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-05-ejercicio> .\build\hotel.exe
FACTURA - Ana Garcia
  Base:  240 EUR
  IVA:   24 EUR
  Total: 264 EUR
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-04-estructura % cd ..
alumno@MacBook-de-alumno pp-lab2-cmake % cd paso-05-ejercicio
alumno@MacBook-de-alumno paso-05-ejercicio % cmake -S . -B build
alumno@MacBook-de-alumno paso-05-ejercicio % cmake --build build
alumno@MacBook-de-alumno paso-05-ejercicio % ./build/hotel
FACTURA - Ana Garcia
  Base:  240 EUR
  IVA:   24 EUR
  Total: 264 EUR
```

Funciona, pero abre `src/main.cpp`: la clase `Factura` está escrita **entera dentro de `main.cpp`** (entre los comentarios `MOVER ESTO` y `HASTA AQUÍ`). Tu tarea es dejarla como las demás: declaración en `include/factura.hpp` e implementación en `src/factura.cpp`. **Al final, el programa debe imprimir exactamente lo mismo.**

5.1 Crear include/factura.hpp (la declaración)

1. En el Explorador de VS Code, haz clic derecho sobre la carpeta `paso-05-ejercicio/include` → **Nuevo archivo...** → escribe `factura.hpp` → Enter.
2. Copia en él la clase desde `main.cpp` y **quita el cuerpo de cada método**: deja solo la primera línea, terminada en `;`. Mira el ejemplo con `getBase`:

ANTES (en main.cpp)

```
double getBase() const { return
reserva_.getTotal(); }
```

DESPUÉS (en factura.hpp)

```
double getBase() const;
```

3. Haz lo mismo con el constructor, `getIva`, `getTotal` e `imprimir`. Los atributos (`private:`) se quedan igual.
4. Arriba del todo añade `#pragma once` y los `#include` de las clases que usa `Factura`:

include/factura.hpp (empieza así)

```
#pragma once
#include "cliente.hpp"
#include "reserva.hpp"

class Factura {
public:
    Factura(const Cliente& cliente, const Reserva& reserva);
    double getBase() const;
    // ... completa tú getIva, getTotal e imprimir ...

private:
    static constexpr double IVA = 0.10;   // IVA de hostelería
    const Cliente& cliente_;
    const Reserva& reserva_;
};
```

5.2 Crear src/factura.cpp (la implementación)

1. Clic derecho sobre `paso-05-ejercicio/src` → **Nuevo archivo...** → `factura.cpp`.
2. Arriba: `#include "factura.hpp"` y `#include <iostream>` (porque `imprimir` usa `std::cout`).
3. Copia cada método **con su cuerpo**, añadiendo `Factura::` delante del nombre. Así:

ANTES (en main.cpp)

```
double getBase() const { return
reserva_.getTotal(); }
```

DESPUÉS (en factura.cpp)

```
double Factura::getBase() const {
    return reserva_.getTotal();
}
```

El constructor, con su lista de inicialización:

src/factura.cpp

```
Factura::Factura(const Cliente& cliente, const Reserva& reserva)
    : cliente_(cliente), reserva_(reserva) {}
```

5.3 Limpiar main.cpp y avisar a CMake

1. En `src/main.cpp`, **borra** toda la clase `Factura` (desde `// ===== MOVER ESTO` hasta `// ===== HASTA AQUÍ`).
2. En su lugar, arriba con los demás `#include`, añade `#include "factura.hpp"`.
3. En `CMakeLists.txt`, añade `src/factura.cpp` a `add_executable` (la regla de oro del paso 4).
4. Guarda **todos** los ficheros (menú **Archivo → Guardar todo**), compila y ejecuta:

Windows · PowerShell

```
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-05-ejercicio> cmake --build build
PS C:\Users\alumno\PP\pp-lab2-cmake\paso-05-ejercicio> .\build\hotel.exe
FACTURA - Ana Garcia
  Base:  240 EUR
  IVA:   24 EUR
  Total: 264 EUR
```

Mac · Terminal

```
alumno@MacBook-de-alumno paso-05-ejercicio % cmake --build build
alumno@MacBook-de-alumno paso-05-ejercicio % ./build/hotel
FACTURA - Ana Garcia
  Base:  240 EUR
  IVA:   24 EUR
  Total: 264 EUR
```

> ✔ **Deberías ver**
>
> La misma factura que al principio: **240 / 24 / 264**. ¡Has separado una clase y la has integrado en CMake!

5.4 Si algo falla

Mensaje	Qué significa	Arreglo
`undefined reference to `Factura::Factura(…)'`	Enlazador: no encuentra el código de `Factura`	Falta `src/factura.cpp` en el `CMakeLists.txt`
`'reserva_' was not declared in this scope` `non-member function … cannot have cv-qualifier`	En `factura.cpp` escribiste un método sin `Factura::` delante	`double Factura::getBase() const`
`'Factura' was not declared in this scope`	`main.cpp` no conoce la clase	Falta `#include "factura.hpp"` en `main.cpp`
`redefinition of 'class Factura'`	La clase está dos veces	No la has borrado de `main.cpp`
`factura.hpp: No such file or directory`	El `.hpp` no está en `include/`	Comprueba en el Explorador dónde lo creaste y que se llama exactamente `factura.hpp`
Compila pero el cambio no se nota	Ficheros sin guardar	**Archivo → Guardar todo**

¿Atascado? La solución completa está en la carpeta `paso-05-solucion`. Compárala con la tuya fichero a fichero.

> **Con Claude**
>
> Si te sale un error que no entiendes, pégale a Claude el **mensaje completo** de `cmake --build build` junto con tu `CMakeLists.txt` y el fichero que crees que falla, y pídele que te explique **qué significa** el error (no solo que te lo arregle).

Para casa · Vuestro proyecto a partir de la plantilla

Ya sabes cómo funciona un proyecto con CMake. Para que no tengáis que montar el vuestro desde cero, hay una **plantilla** con todo preparado: la estructura `include/` + `src/`, el `CMakeLists.txt`, los **tests con GoogleTest** y el **pipeline de CI** de GitHub Actions (lo que pide la Entrega 2). Está en **https://github.com/jcor-cunpir/pp-plantilla-proyecto**.

> **Atención**
>
> Recordad: el repositorio de la **Entrega 2** tiene que estar compartido con el profesor antes de las **23:59 del sábado 14 de noviembre**, con el pipeline en verde y al menos 5 tests que pasen. Empezar desde la plantilla os da todo eso funcionando desde el primer día.

A. Crear el repositorio del grupo (lo hace UNA persona del grupo)

1. Entra en **https://github.com/jcor-cunpir/pp-plantilla-proyecto** con tu cuenta de GitHub.
2. Pulsa el botón verde **Use this template** → **Create a new repository**.
3. En *Repository name* escribe el nombre de vuestro proyecto (sin espacios, p. ej. `gestor-biblioteca`). Elige **Private** y pulsa **Create repository**.
4. En vuestro repositorio nuevo: **Settings → Collaborators → Add people**, y añade a cada miembro del grupo y al profesor (su usuario de GitHub es `jcor-cunpir`).

B. Descargarlo y compilarlo (cada miembro del grupo)

Windows · PowerShell

```
PS C:\Users\alumno\PP> git clone https://github.com/vuestro-usuario/gestor-biblioteca.git
PS C:\Users\alumno\PP> cd gestor-biblioteca
PS C:\Users\alumno\PP\gestor-biblioteca> cmake -S . -B build -G Ninja
PS C:\Users\alumno\PP\gestor-biblioteca> cmake --build build
PS C:\Users\alumno\PP\gestor-biblioteca> .\build\gestor.exe
Proyecto compilado con CMake. Habitacion de ejemplo: 101
Reservada
```

Mac · Terminal

```
alumno@MacBook-de-alumno PP % git clone https://github.com/vuestro-usuario/gestor-biblioteca.g
it
alumno@MacBook-de-alumno PP % cd gestor-biblioteca
alumno@MacBook-de-alumno gestor-biblioteca % cmake -S . -B build
alumno@MacBook-de-alumno gestor-biblioteca % cmake --build build
alumno@MacBook-de-alumno gestor-biblioteca % ./build/gestor
Proyecto compilado con CMake. Habitacion de ejemplo: 101
Reservada
```

La primera vez que configuras, CMake **descarga GoogleTest** de internet: tarda un poco más (hace falta conexión). Las siguientes veces, no.

> **✘ Si en vez de eso ves…**
>
> `Failed to clone repository: 'https://github.com/google/googletest.git'`, o un error que dice que no encuentra `git`: CMake usa **Git** para descargar GoogleTest. Comprueba que tienes conexión y que `git --version` responde (si no, **Anexo, paso W4 o M1**). Después borra la carpeta `build` y vuelve a configurar.

C. Ejecutar los tests

Windows · PowerShell

```
PS C:\Users\alumno\PP\gestor-biblioteca> cd build
PS C:\Users\alumno\PP\gestor-biblioteca\build> ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests .....................   Passed    0.02 sec

100% tests passed, 0 tests failed out of 1
PS C:\Users\alumno\PP\gestor-biblioteca\build> .\tests_gestor.exe
[==========] Running 5 tests from 1 test suite.
…
[  PASSED  ] 5 tests.
PS C:\Users\alumno\PP\gestor-biblioteca\build> cd ..
```

Mac · Terminal

```
alumno@MacBook-de-alumno gestor-biblioteca % cd build
alumno@MacBook-de-alumno build % ctest --output-on-failure
    Start 1: GestorTests
1/1 Test #1: GestorTests .....................   Passed    0.01 sec

100% tests passed, 0 tests failed out of 1
alumno@MacBook-de-alumno build % ./tests_gestor
[==========] Running 5 tests from 1 test suite.
…
[  PASSED  ] 5 tests.
alumno@MacBook-de-alumno build % cd ..
```

D. Comprobar el pipeline (CI) en GitHub

1. En la página de vuestro repositorio, pestaña **Actions**: veréis una ejecución llamada **CI**.
2. Cuando termine debe salir con un ✔ **verde**: GitHub ha compilado el proyecto en una máquina Linux limpia y ha pasado los tests.
3. Cada `git push` lo vuelve a lanzar. Si sale una ✘ roja, haced clic en ella para ver qué paso ha fallado.

E. Empezar a poner vuestro código

- Vuestras clases: el `.hpp` en `include/`, el `.cpp` en `src/`, y **cada `.cpp` nuevo en la lista `FUENTES`** del `CMakeLists.txt` (está marcada con `>>> AQUÍ <<<`).
- Vuestros tests: un fichero `tests/test_<clase>.cpp` por clase, añadido a `add_executable(tests_gestor …)`. Los tests de ejemplo os sirven de modelo.
- Cuando tengáis vuestras clases, cambiad `src/main.cpp` para que las use en lugar de `Habitacion` y después borrad la clase de ejemplo (su `.hpp`, su `.cpp`, su test y sus líneas en el `CMakeLists.txt`).
- Commit y push como en la práctica de Git: ramas, Conventional Commits y Pull Requests.

Resumen: la chuleta de la práctica

Quiero...	Windows (PowerShell)	Mac (Terminal)
Saber dónde estoy / qué hay	`pwd` · `ls`	`pwd` · `ls`
Moverme de carpeta	`cd carpeta` · `cd ..`	`cd carpeta` · `cd ..`
Compilar a mano	`g++ main.cpp otra.cpp -o hotel`	`g++ main.cpp otra.cpp -o hotel`
Ejecutar lo compilado a mano	`.\hotel.exe`	`./hotel`
Configurar con CMake (1.ª vez)	`cmake -S . -B build -G Ninja`	`cmake -S . -B build`
Compilar con CMake (cada vez)	`cmake --build build`	`cmake --build build`
Ejecutar	`.\build\hotel.exe`	`./build/hotel`
Pasar los tests	`cd build` + `ctest --output-on-failure`	`cd build` + `ctest --output-on-failure`
Empezar de cero (borrar build)	`Remove-Item -Recurse -Force build`	`rm -rf build`

Si algo falla (los errores más frecuentes)

Ves esto	Qué pasa	Qué hacer
`No such file or directory` / `does not appear to contain CMakeLists.txt`	Estás en otra carpeta	`pwd` y `ls`; muévete con `cd`
`'g++'` / `'cmake'` `no se reconoce como nombre de un cmdlet`	Windows no encuentra el programa	Anexo, pasos W1-W3 (el PATH es el W3); cierra y abre la terminal
`zsh: command not found: cmake`	Mac no encuentra CMake	Anexo, paso M2 (*How to Install For Command Line Use*)
`zsh: command not found: code`	Falta el comando de VS Code (Mac)	`⌘⇧P` → *Shell Command: Install 'code' command in PATH*
`unable to find a build program corresponding to "Ninja"`	Falta Ninja (Windows)	`pacman -S --needed mingw-w64-x86_64-ninja` en MSYS2
`Does not match the generator used previously`	`build` se creó con otra configuración	Borra `build` y vuelve a configurar
`undefined reference to …` / `Undefined symbols`	Enlazador: falta el código de algo	¿Está el `.cpp` en el `CMakeLists.txt`? ¿Está definido el método o el `static`?
`fichero.cpp:8:38: error: …`	Compilador: error en esa línea	Ve a esa línea y columna
He cambiado el código y no cambia nada	No has guardado	● en la pestaña = sin guardar → `Ctrl+S`

Anexo · Instalar las herramientas (repaso de la Práctica 0)

Solo si en el paso 1.1 te falta alguna herramienta. Necesitas **conexión a internet** y unos **15-20 minutos**. Haz solo la parte de tu sistema y, dentro de ella, solo los pasos de lo que te falte. Al terminar, **cierra todas las terminales, abre una nueva** y repite las comprobaciones del paso 1.1.

Herramienta	Windows	Mac
Compilador `g++`	MSYS2 → **W1-W3**	Xcode Command Line Tools → **M1**
`cmake`	MSYS2 → **W2-W3**	cmake.org → **M2**
`ninja`	MSYS2 → **W2-W3**	no hace falta
`git`	git-scm.com → **W4**	viene con las Command Line Tools → **M1**
VS Code	code.visualstudio.com → **W5**	code.visualstudio.com → **M3**

Windows

W1. Instalar MSYS2 (el "gestor de paquetes" desde el que se instalan g++, CMake y Ninja):

1. Entra en **https://www.msys2.org** y pulsa el enlace del instalador, `msys2-x86_64-XXXXXXXX.exe` (la fecha del nombre cambia).
2. Ábrelo desde *Descargas* y pulsa **Siguiente** en todo, **sin cambiar la carpeta**: tiene que ser `C:\msys64`.
3. Al final, deja marcada **Run MSYS2 now** y pulsa **Finalizar**. Se abre una terminal **distinta** de PowerShell: la de MSYS2.

W2. Instalar g++, CMake y Ninja. En la terminal **MSYS2 MSYS** (tecla Windows → escribe MSYS2 → "MSYS2 MSYS"), primero actualiza:

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -Syu
```

Si pregunta `Proceed with installation? [Y/n]`, escribe `Y` y Enter. Si la ventana se cierra sola, es normal: ábrela otra vez y repite `pacman -Syu` hasta que diga `there is nothing to do`. Después instala las tres herramientas de una vez (confirma con `Y`):

Windows · terminal MSYS2 MSYS (no PowerShell)

```
$ pacman -S --needed mingw-w64-x86_64-gcc mingw-w64-x86_64-cmake mingw-w64-x86_64-ninja
```

W3. Añadir la carpeta de MSYS2 al PATH (para que PowerShell encuentre `g++`, `cmake` y `ninja`):

1. Tecla Windows → escribe **variables de entorno** → **Editar las variables de entorno del sistema** → botón **Variables de entorno...**
2. En la mitad de arriba (*Variables de usuario*), selecciona **Path** → **Editar...** → **Nuevo** → escribe exactamente `C:\msys64\mingw64\bin`
3. **Aceptar** en las tres ventanas.
4. **Cierra todas las ventanas de PowerShell** (y VS Code, si estaba abierto): solo las terminales nuevas ven el cambio.

W4. Instalar Git: entra en **https://git-scm.com/downloads/win**, pulsa **Click here to download**, abre el instalador y pulsa **Next** en todo (las opciones por defecto valen).

W5. Instalar VS Code (si no lo tienes): **https://code.visualstudio.com** → **Download for Windows**. Durante la instalación, deja marcada la casilla **Añadir a la RUTA (PATH)**.

> ✔ **Deberías ver**
> En una PowerShell **nueva**, `g++ --version`, `cmake --version`, `ninja --version` y `git --version` responden con un número de versión (como en el paso 1.1).

> ✘ **Si en vez de eso ves…**
> Siguen sin reconocerse: comprueba en W3 que la ruta está escrita **exactamente** como `C:\msys64\mingw64\bin` (sin espacios al final) y que has abierto una terminal nueva. Si aun así falla, reinicia el ordenador.
> En W2, `error: target not found`: hay una errata en el nombre de algún paquete. Copia el comando tal cual.

Mac

M1. Instalar g++ y Git (Xcode Command Line Tools). En la Terminal:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % xcode-select --install
```

Se abre una ventana: pulsa **Instalar**, acepta la licencia y espera (entre 5 y 15 minutos). Si en lugar de eso ves `command line tools are already installed`, ya los tienes: pasa a M2.

M2. Instalar CMake:

1. Entra en **https://cmake.org/download/**. En *Binary distributions*, descarga el fichero de macOS que termina en `macos-universal.dmg` (vale para chips Intel y Apple M).
2. Abre el `.dmg` y arrastra el icono de **CMake** a la carpeta **Applications**.
3. Abre **CMake** desde Aplicaciones (si macOS pregunta si quieres abrirla, pulsa **Abrir**).
4. **Este es el paso que más se olvida:** en el menú de arriba, **Tools → How to Install For Command Line Use**. Copia el comando que aparece, que es este:

Mac · Terminal

```
alumno@MacBook-de-alumno ~ % sudo "/Applications/CMake.app/Contents/bin/cmake-gui" --install
Password:
```

Pégalo en la Terminal y pulsa Enter. Te pide la contraseña de tu Mac: escríbela (**no se ve nada mientras escribes**, es normal) y pulsa Enter. Cierra la Terminal, abre una nueva y comprueba con `cmake --version`.

> ✘ **Si en vez de eso ves…**
> `Failed to create symlink` o `No such file or directory` con `/usr/local/bin`: esa carpeta no existe en tu Mac. Créala con `sudo mkdir -p /usr/local/bin` y repite el comando `sudo "/Applications/CMake.app/…" --install`.
> Si usas **Homebrew**, también vale `brew install cmake` en lugar de los pasos 1-4.

M3. Instalar VS Code (si no lo tienes): **https://code.visualstudio.com** → **Download for Mac**, arrastra la aplicación a **Aplicaciones** y ábrela. Después, para poder usar `code .`: ⌘ Cmd + ⇧ Shift + P → **Shell Command: Install 'code' command in PATH**.

> **✔ Deberías ver**
> En una Terminal **nueva**, `g++ --version`, `cmake --version` y `git --version` responden con un número de versión (como en el paso 1.1).